

SIMATS ENGINEERING (SAVEETHA SCHOOL OF ENGINEERING)

Department of Computer Science and Engineering

ASSESSMENT TOOL 1 – Constraint-Based Problem Solving

Course Code:	CSA06	Course Title:	Design and Analysis of Algorithms
CO Assessed:	CO5– Naturalization (BL5,BL6)	Assessment:	Assessment Tool 1 – Constraint Based Problem Solving
Weightage:	30% of CIA	Total Marks:	100
CO5 Statement:	<i>Analyze computational problems using backtracking techniques and complexity classes such as P, NP, NP-Hard, and NP-Complete for combinatorial problem solving.</i>		
Student Name:	G.Srivardhan	Reg. No.:	192425208
Section / Batch:		Date:	

Instructions to Students

- Answer the following question. Total Marks: 100.
- Apply backtracking techniques systematically for combinatorial problem solving.
- Clearly classify problems under P, NP, NP-Hard, and NP-Complete categories wherever required.
- Provide proper justification, state-space representation, and complexity analysis for all solutions.
- Neat presentation and logical explanation are mandatory.

Question Type	Focus Area	Questions
Constraint Based Problem Solving	Analyze combinatorial problems using backtracking and complexity classes such as P, NP, NP-Hard, and NPComplete.	Q1

SECTION A – CONSTRAINT-BASED PROBLEM SOLVING

1) AIM

To schedule tasks with given execution times and deadlines to maximize the number of tasks completed, using a backtracking-based solution with pruning (discarding infeasible branches and using branch-and-bound to reduce unnecessary exploration).

2) PROCEDURE

1. Read n tasks. For each task i , store:
 - duration $t[i]$
 - deadline $d[i]$
2. Use backtracking to build the schedule step-by-step from time 0.
 - Maintain `curr_time` (time spent so far).

- Maintain chosen (list of tasks included so far).
 - Maintain used[i] to avoid scheduling the same task twice.
3. At each recursion level:
 - For every unused task i, compute $\text{finish_time} = \text{curr_time} + \text{duration}[i]$.
 - If $\text{finish_time} \leq \text{deadline}[i]$, then the task is feasible—include it and recurse.
 4. Pruning (to reduce search space):
 - Deadline pruning: if a task misses its deadline, skip that branch.
 - Bound pruning: if $\text{curr_count} + \text{remaining} \leq \text{best_count}$, stop exploring because it cannot beat the current best.
 5. Track the best answer (maximum tasks and one corresponding schedule).
-

3) CODE (Python)

```
from typing import List, Tuple
```

```
def max_tasks_with_deadlines(tasks: List[Tuple[int, int]]) -> Tuple[int, List[int]]:
    n = len(tasks)
    durations = [t for t, d in tasks]
    deadlines = [d for t, d in tasks]
```

```
    best_count = 0
    best_schedule = []
    used = [False] * n
```

```
    def backtrack(curr_time: int, chosen: List[int], curr_count: int):
        nonlocal best_count, best_schedule
```

```
        remaining = n - len(chosen)
```

```
        # Pruning by upper bound
        if curr_count + remaining <= best_count:
            return
```

```
        # Update best answer
        if curr_count > best_count:
            best_count = curr_count
            best_schedule = chosen[:]
```

```
        # Try adding each unused feasible task
        for i in range(n):
            if not used[i]:
                finish_time = curr_time + durations[i]
                if finish_time <= deadlines[i]:
                    used[i] = True
                    chosen.append(i)
                    backtrack(finish_time, chosen, curr_count + 1)
                    chosen.pop()
                    used[i] = False
```

```
    backtrack(0, [], 0)
```

```
return best_count, best_schedule
```

```
if __name__ == "__main__":  
    tasks = [(2, 4), (1, 3), (3, 5), (2, 6)]  
    count, schedule = max_tasks_with_deadlines(tasks)  
    print(count)  
    print(schedule)
```

output

The screenshot shows a web-based coding assessment interface. At the top, it says 'ASSESSMENT- CO5-AT1-Constraint-Based Problem Solving'. Below this, there are tabs for 'Python', 'C', 'C++', and 'Java', with 'Python' selected. On the left, there's a 'QUESTIONS' sidebar with a list of questions, including 'Q1 Scheduling with Deadlines (NP-Hard Constraint Problem)' which is marked as '100 marks' and 'Q1 answered'. The main area displays the question title 'Q1 Scheduling with Deadlines (NP-Hard Constraint Problem)' and a description: 'Schedule tasks with deadlines to maximize completed tasks. Clearly define constraints such as deadlines and execution times. Analyze how constraints influence scheduling decisions. Develop a backtracking-based solution. Apply pruning to discard infeasible schedules. Justify correctness and discuss computational complexity.' Below the question, there's a 'Your Code' section with a dark-themed code editor containing Python code for a backtracking solution. To the right of the code editor, there's an 'Input' section with a text area labeled 'stdin input' and an 'Output' section with a text area showing the output '3' and '[0, 1, 3]'.

```
1 from typing import List, Tuple  
2  
3 def max_tasks_with_deadlines(tasks: List[Tuple[int, int]]) -> Tuple[int, List[int]]:  
4     n = len(tasks)  
5     durations = [t for t, d in tasks]  
6     deadlines = [d for t, d in tasks]  
7  
8     best_count = 0  
9     best_schedule = []  
10    used = [False] * n  
11  
12    def backtrack(curr_index: int, chosen: List[int], curr_count: int):  
13        # Pruning: If current count is less than or equal to best count, skip  
14        remaining = n - len(chosen)  
15        if curr_count + remaining <= best_count:  
16            return  
17  
18        if curr_count > best_count:  
19            best_count = curr_count  
20            best_schedule = chosen[:]  
21  
22    backtrack(0, [], 0)
```

4) RESULT

For the given tasks [(2,4), (1,3), (3,5), (2,6)], the maximum number of tasks that can be completed is 3.

One optimal schedule is [0, 1, 3].

Code of Conduct

I _____ (Name / Reg No) certify that this submission is my original work and that I have adhered to the guidelines specified for this assessment. I understand that any violation of academic integrity rules will result in disciplinary action.

Signature of the Student: _____

RUBRICS FOR CALCULATION

Criteria	Wt.	Level 4 – Exemplary	Level 3 – Proficient	Level 2 – Developing	Level 1 – Beginning
Problem Understanding	25%	Clearly defines the problem and identifies all relevant constraints accurately	Identifies most constraints with minor gaps	Partial understanding; misses key constraints	Fails to identify constraints correctly
Constraint Analysis	25%	Analyzes constraints effectively and prioritizes them logically	Provides reasonable analysis with some prioritization	Limited analysis; weak prioritization	No meaningful analysis or prioritization
Solution Development	25%	Develops optimal and feasible solutions satisfying all constraints	Develops workable solutions with minor limitations	Solutions partially satisfy constraints	Solutions do not meet constraints
Application of Concepts	15%	Effectively applies appropriate concepts, methods, or tools	Applies concepts with minor errors	Limited application; requires guidance	Unable to apply relevant concepts
Justification	10%	Provides strong justification for decisions with logical reasoning	Provides justification with some clarity	Weak or unclear justification	No justification provided

Marks Evaluation:

Criteria	Wt.	Level 4 – Exemplary	Level 3 – Proficient	Level 2 – Developing	Level 1 – Beginning
Problem Understanding	25%				
Constraint Analysis	25%				
Solution Development	25%				
Application of Concepts	15%				
Justification	10%				

Total Marks (100):	
---------------------------	--

Faculty In-charge	Course Coordinator	Head of Department
Name: _____	Name: _____	Name: _____
Signature: _____	Signature: _____	Signature: _____
Date: _____	Date: _____	Date: _____